Xây dựng mô hình YOLOv3

1. Tổng quan nguyên lý hoạt động

YOLOv3 tiếp cận bài toán phát hiện đối tượng theo hướng hồi quy trực tiếp. Thay vì xử lý từng vùng ảnh riêng biệt, mô hình chia ảnh thành các ô lưới và dự đoán bounding box cùng nhãn lớp ngay trên từng ô.

Trong hiện thực này, mô hình trả về ba đầu ra với kích thước khác nhau:

- 13×13: phù hợp với vật thể lớn
- 26×26: phù hợp với vật thể trung bình
- 52×52: phù hợp với vật thể nhỏ

Cách thiết kế đa tỉ lệ này giúp cải thiện đáng kể khả năng phát hiện so với các phiên bản YOLO trước đó.

2. ConvBlock

ConvBlock là đơn vị cấu trúc lặp lại xuyên suốt toàn bộ mô hình. Mỗi block bao gồm một lớp tích chập, theo sau là Batch Normalization và hàm kích hoạt LeakyReLU.

```python
class ConvBlock(nn.Module):
    def __init__(self, in_c, out_c, kernel_size=1, stride=1, bn_act=True):
        super().__init__()
        padding = kernel_size // 2
        self.bn_act = bn_act

        self.conv = nn.Conv2d(
            in_c,
            out_c,
            kernel_size,
            stride,
            padding,
            bias=not bn_act
        )

        if bn_act:
            self.bn = nn.BatchNorm2d(out_c)
            self.act = nn.LeakyReLU(0.1)

    def forward(self, x):
        if self.bn_act: return self.act(self.bn(self.conv(x)))
        else: return self.conv(x)
```

Hình 1. Cấu trúc của ConvBlock

Một chi tiết quan trọng là việc vô hiệu hóa bias trong lớp convolution khi sử dụng BatchNorm. Điều này là hợp lý vì BatchNorm đã đảm nhiệm vai trò điều chỉnh dịch chuyển trung bình.

Việc sử dụng padding bằng một nửa kích thước kernel giúp giữ nguyên kích thước không gian của feature map trong các phép tích chập thông thường

3. ResidualBlock

ResidualBlock được xây dựng dựa trên ý tưởng của ResNet, cho phép mạng học phần dư thay vì ánh xạ trực tiếp. Cấu trúc này giúp giảm hiện tượng mất gradient khi huấn luyện mạng sâu.[1]

```
class ResidualBlock(nn.Module):
    def __init__(self, channels):
        super().__init__()
        self.layer1 = ConvBlock(channels, channels//2, kernel_size=1, stride=1)
        self.layer2 = ConvBlock(channels//2, channels, kernel_size=3, stride=1)

    def forward(self, x):
        return x + self.layer2(self.layer1(x))
```

Hình 2. Cấu trúc của Residual Block

Về mặt toán học, đầu ra của block có dạng:

$$output = x + F(x)$$

Trong đó F(x) là chuỗi các phép biến đổi gồm một convolution 1×1 để giảm số kênh, sau đó là convolution 3×3 để học đặc trưng không gian. Thiết kế này vừa giảm chi phí tính toán vừa đảm bảo khả năng biểu diễn.

4. Backbone Darknet-53

Darknet-53 đóng vai trò trích xuất đặc trưng từ ảnh đầu vào. Mạng bao gồm nhiều tầng convolution kết hợp với residual block, tạo thành một kiến trúc sâu.

```python
class Darknet53(nn.Module):
    def __init__(self):
        super().__init__()

        self.layer1 = ConvBlock(3, 32, 3, 1)
        self.layer2 = ConvBlock(32, 64, 3, 2)
        self.res1 = self._make_layer(64, 1)

        self.layer3 = ConvBlock(64, 128, 3, 2)
        self.res2 = self._make_layer(128, 2)

        self.layer4 = ConvBlock(128, 256, 3, 2)
        self.res3 = self._make_layer(256, 8)

        self.layer5 = ConvBlock(256, 512, 3, 2)
        self.res4 = self._make_layer(512, 8)

        self.layer6 = ConvBlock(512, 1024, 3, 2)
        self.res5 = self._make_layer(1024, 4)
```

Hình 3. Cấu trúc của Darknet53

	Type	Filters	Size	Output
	Convolutional	32	3 × 3	256 × 256
	Convolutional	64	3 × 3 / 2	128 × 128
1×	Convolutional	32	1 × 1	
	Convolutional	64	3 × 3	
	Residual			128 × 128
	Convolutional	128	3 × 3 / 2	64 × 64
2×	Convolutional	64	1 × 1	
	Convolutional	128	3 × 3	
	Residual			64 × 64
	Convolutional	256	3 × 3 / 2	32 × 32
8×	Convolutional	128	1 × 1	
	Convolutional	256	3 × 3	
	Residual			32 × 32
	Convolutional	512	3 × 3 / 2	16 × 16
8×	Convolutional	256	1 × 1	
	Convolutional	512	3 × 3	
	Residual			16 × 16
	Convolutional	1024	3 × 3 / 2	8 × 8
4×	Convolutional	512	1 × 1	
	Convolutional	1024	3 × 3	
	Residual			8 × 8
	Avgpool		Global	
	Connected		1000	
	Softmax			

Hình 4. Cấu trúc trong paper [2]

Mỗi lần sử dụng stride bằng 2, kích thước không gian của feature map bị giảm đi một nửa, trong khi số kênh tăng lên. Điều này giúp mô hình dần chuyển từ biểu diễn chi tiết sang biểu diễn mang tính ngữ nghĩa cao.

Đáng chú ý, mô hình không chỉ sử dụng output cuối cùng mà trích xuất ba mức đặc trưng:

- Feature mức nông (52×52): giàu thông tin vị trí
- Feature trung gian (26×26)
- Feature sâu (13×13): giàu thông tin ngữ nghĩa

Các mức đặc trưng này là cơ sở để xây dựng cơ chế đa tỉ lệ.

```python
def forward(self, x):
    x = self.layer1(x)
    x = self.layer2(x)
    x = self.res1(x)

    x = self.layer3(x)
    x = self.res2(x)

    x = self.layer4(x)
    feat_small = self.res3(x)   # 52x52

    x = self.layer5(feat_small)
    feat_medium = self.res4(x)  # 26x26

    x = self.layer6(feat_medium)
    feat_large = self.res5(x)   # 13x13

    return feat_small, feat_medium, feat_large
```

Hình 5. Darknet53 trả về 3 mức đặc trưng

5. DetectionBlock

DetectionBlock chịu trách nhiệm chuyển đổi feature map thành các dự đoán cụ thể.

```python
class DetectionBlock(nn.Module):
    def __init__(self, in_channels, hidden_channels, num_classes):
        super().__init__()

        self.layers = nn.Sequential(
            ConvBlock(in_channels, hidden_channels, 1),
            ConvBlock(hidden_channels, hidden_channels * 2, 3),
            ConvBlock(hidden_channels*2, hidden_channels, 1),
            ConvBlock(hidden_channels, hidden_channels * 2, 3),
            ConvBlock(hidden_channels*2, hidden_channels, 1),
        )

        self.pred_conv = nn.Sequential(
            ConvBlock(hidden_channels, hidden_channels * 2, 3),
            ConvBlock(
                hidden_channels * 2,
                3 * (num_classes + 5),
                kernel_size=1,
                bn_act=False
            )
        )

        self.num_classes = num_classes
```

Hình 6. Cấu trúc của Dectection Block

Trước khi dự đoán, feature được xử lý qua nhiều lớp convolution xen kẽ giữa kernel 1×1 và 3×3. Cách thiết kế này cho phép kết hợp thông tin giữa các kênh và khai thác cấu trúc không gian hiệu quả hơn.

Lớp cuối cùng tạo ra tensor với số kênh:

$$3 \times (num_classes + 5)$$

```python
def forward(self, x):
    x = self.layers(x)
    route = x   # giữ feature map cho FPN
    pred = self.pred_conv(x)

    B, _, H, W = pred.shape
    pred = pred.reshape(B, 3, self.num_classes + 5, H, W)
    pred = pred.permute(0, 1, 3, 4, 2)

    return route, pred
```

Hình 7. Minh hoạ đầu ra của predict

Trong đó:

- 3 là số anchor box trên mỗi ô
- 5 đại diện cho tọa độ và độ tin cậy của bounding box

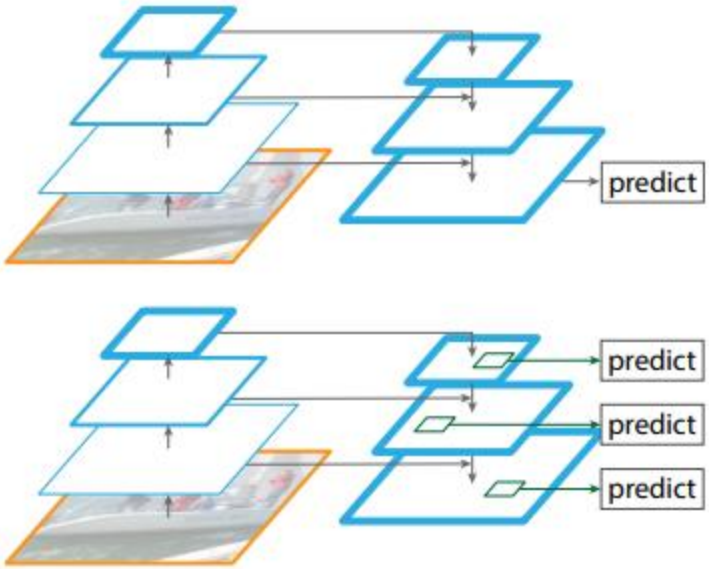
Sau khi biến đổi tensor (reshape và permute), đầu ra có dạng:

$$(B, 3, H, W, num_classes + 5)$$

Điều này có nghĩa là mỗi ô lưới dự đoán ba bounding box độc lập.

6. Feature Pyramid Network (FPN)

Một thành phần quan trọng của YOLOv3 là việc kết hợp thông tin từ nhiều mức đặc trưng khác nhau thông qua FPN.



Hình 8. Minh hoạ kiến trúc FPN[3]

Quá trình này diễn ra bằng cách:

- Lấy feature từ tầng sâu
- Upsample để tăng kích thước
- Ghép (concatenate) với feature từ tầng nông hơn có cùng kích thước

Việc kết hợp này giúp giữ được cả thông tin ngữ nghĩa (từ tầng sâu) và thông tin vị trí (từ tầng nông), từ đó cải thiện khả năng phát hiện vật thể nhỏ.

Tài liệu tham khảo

[1] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," Dec. 10, 2015, *arXiv*: arXiv:1512.03385. doi: 10.48550/arXiv.1512.03385.

[2] J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement," Apr. 08, 2018, *arXiv*: arXiv:1804.02767. doi: 10.48550/arXiv.1804.02767.

[3] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature Pyramid Networks for Object Detection," Apr. 19, 2017, *arXiv*: arXiv:1612.03144. doi: 10.48550/arXiv.1612.03144.